

## **Trabajo Práctico Integrador**

# **Base de Datos II**

### **Food Store**

#### **Integrantes:**

RAMIRO QUIROGA  
SANTIAGO COPIA  
FRANCO GAGLIARDI  
IGNACE CEBALLOS  
EMMANUEL MIRANDA  
CIRO CATTANEO

## **Índice**

### **1. Marco teórico**

#### **1.1 El Modelo Relacional**

El modelo relacional estructura la información en relaciones (tablas) compuestas por atributos (columnas) y tuplas (filas). Cada tabla representa una entidad del dominio (categoria, producto, usuario, pedido, detalle\_pedido). Los vínculos entre entidades se establecen de forma matemática mediante la correspondencia de valores entre claves primarias (PK) y claves foráneas (FK), eliminando la necesidad de punteros físicos.

#### **1.2 Normalización**

La normalización es el proceso formal para estructurar un esquema relacional reduciendo la redundancia y evitando anomalías de inserción, actualización y borrado:

- Primera Forma Normal (1FN): Requiere que todos los atributos sean atómicos e indivisibles. En el dominio, se satisface al abstraer la lista de ítems de un pedido hacia la entidad `detalle_pedido`.
- Segunda Forma Normal (2FN): Exige estar en 1FN y que ningún atributo no clave dependa parcialmente de una clave primaria compuesta. Al implementarse claves primarias surrogadas simples (`id`), la 2FN se satisface de manera directa.
- Tercera Forma Normal (3FN): Elimina dependencias transitivas entre atributos no clave. Los nombres de categorías o precios del catálogo no residen en el pedido, sino que se vinculan por claves foráneas.

### 1.3 Integridad Referencial

La integridad referencial asegura que toda clave foránea mantenga correspondencia con una clave primaria válida en la tabla referenciada. El motor de base de datos aplica estas reglas mediante restricciones declarativas (`PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `CHECK`, `UNIQUE`).

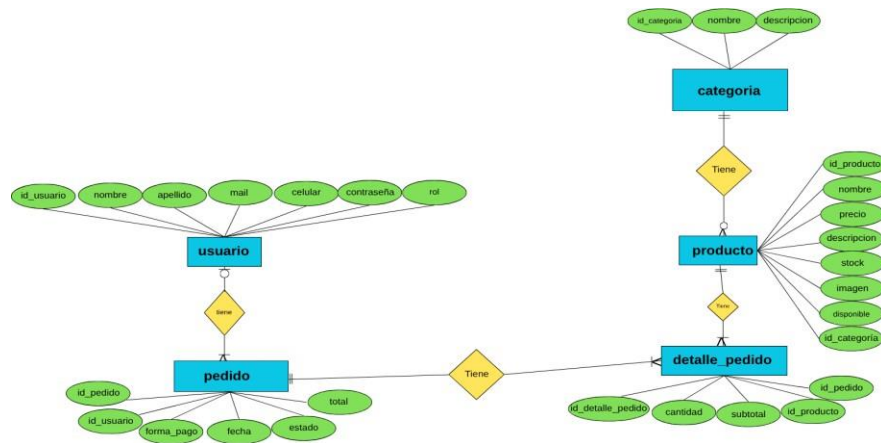
### 1.4 Introducción a Transacciones y Propiedades ACID

Una transacción es una unidad lógica de trabajo que agrupa una o más sentencias SQL delimitadas por `BEGIN`, `COMMIT` o `ROLLBACK`. Para preservar la consistencia de los datos, el motor implementa las propiedades ACID:

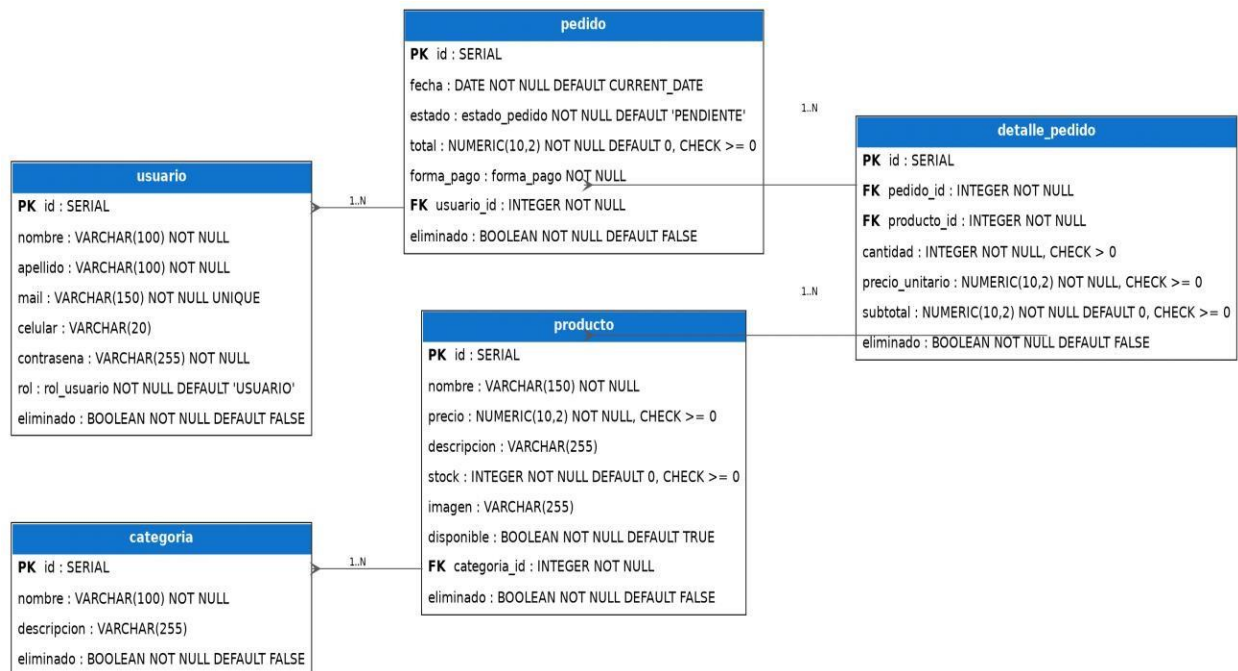
- Atomicidad: Se ejecutan todas las operaciones de la transacción o no se ejecuta ninguna.
- Consistencia: La transacción traslada la base de datos de un estado válido a otro estado válido acorde a sus restricciones.
- Aislamiento: Las operaciones concurrentes no generan interferencias mutuas según el nivel de aislamiento configurado.
- Durabilidad: Una vez confirmados (`COMMIT`), los cambios persisten de manera permanente frente a fallos del sistema.

## 2. Diagramas

### 2.1 Diagrama Entidad-Relación (DER) — modelo conceptual



## 2.2 Esquema relacional / físico



## 3. Decisiones de diseño

### 3.1 Normalización (1FN a BCNF)

- 1FN: los productos de un pedido no se guardan como lista dentro de pedido, sino como filas separadas en detalle\_pedido. Así cada dato queda atómico (un valor por celda).
- 2FN: todas las tablas usan id como clave única (no compuesta), así que 2FN se cumple sola: no hay forma de que algo dependa de "una parte" de la clave.
- 3FN: no se repiten datos que ya están en otra tabla. Ej: producto guarda categoria\_id, no el nombre de la categoría (eso evita tener que actualizar muchas filas si el nombre cambia).
- BCNF: en usuario hay dos claves candidatas (id y mail, porque mail es UNIQUE) y ambas determinan el resto de los datos, así que está bien. La única excepción son subtotal y total: se calculan a partir de otros campos (cantidad × precio, o la suma de subtotales), así que técnicamente son datos repetidos. Se aceptan igual porque los mantiene un trigger automáticamente (ver punto 3.2), nunca se pueden desincronizar a mano.

### 3.2 Manejo del total

Se eligió guardar total en la tabla pedido (no calcularlo cada vez con una consulta) y mantenerlo actualizado con un trigger cada vez que cambian los detalles del pedido.

¿Por qué? Porque los pedidos se listan mucho más seguido de lo que se crean. Si el total no estuviera guardado, cada listado tendría que sumar todos los detalles de nuevo. Guardarlo lo hace instantáneo, y el trigger garantiza que nunca quede desactualizado (no depende de que alguien se acuerde de recalcularlo).

Por el mismo motivo, precio\_unitario se copia en el detalle en vez de leer el precio actual del producto: si el precio cambia después, los pedidos viejos no se alteran.

### 3.3 Borrado lógico (soft delete)

Ninguna tabla borra filas de verdad. En vez de DELETE, se usa una columna eliminado (booleano) que se pone en TRUE. Todas las tablas repiten esta misma columna, en vez de heredarla de una tabla base, porque en PostgreSQL eso complica las claves foráneas y las consultas sin ganar mucho a cambio.

- Consultas: las vistas (v\_productos\_vigentes, etc.) ya filtran eliminado = FALSE, así no hay que acordarse de ponerlo en cada consulta.
- Índices: se usan índices parciales (solo sobre las filas no eliminadas), para que sean más chicos y rápidos.
- Historial: como nada se borra de verdad, los pedidos viejos de un usuario o producto eliminado se siguen pudiendo consultar.

### 3.4 Índices

Regla general: se indexa una columna cuando se la usa seguido en WHERE, JOIN u ORDER BY. Indexar de más solo hace las escrituras más lentas sin necesidad.

## 4. Capturas de ejecución y análisis de índices (EXPLAIN ANALYZE)

## 4.1 Consulta por usuario\_id en pedido

Antes de crear el índice (Seq Scan):

The screenshot shows a PostgreSQL client interface with a script editor and a results panel. The script editor contains the following SQL commands:

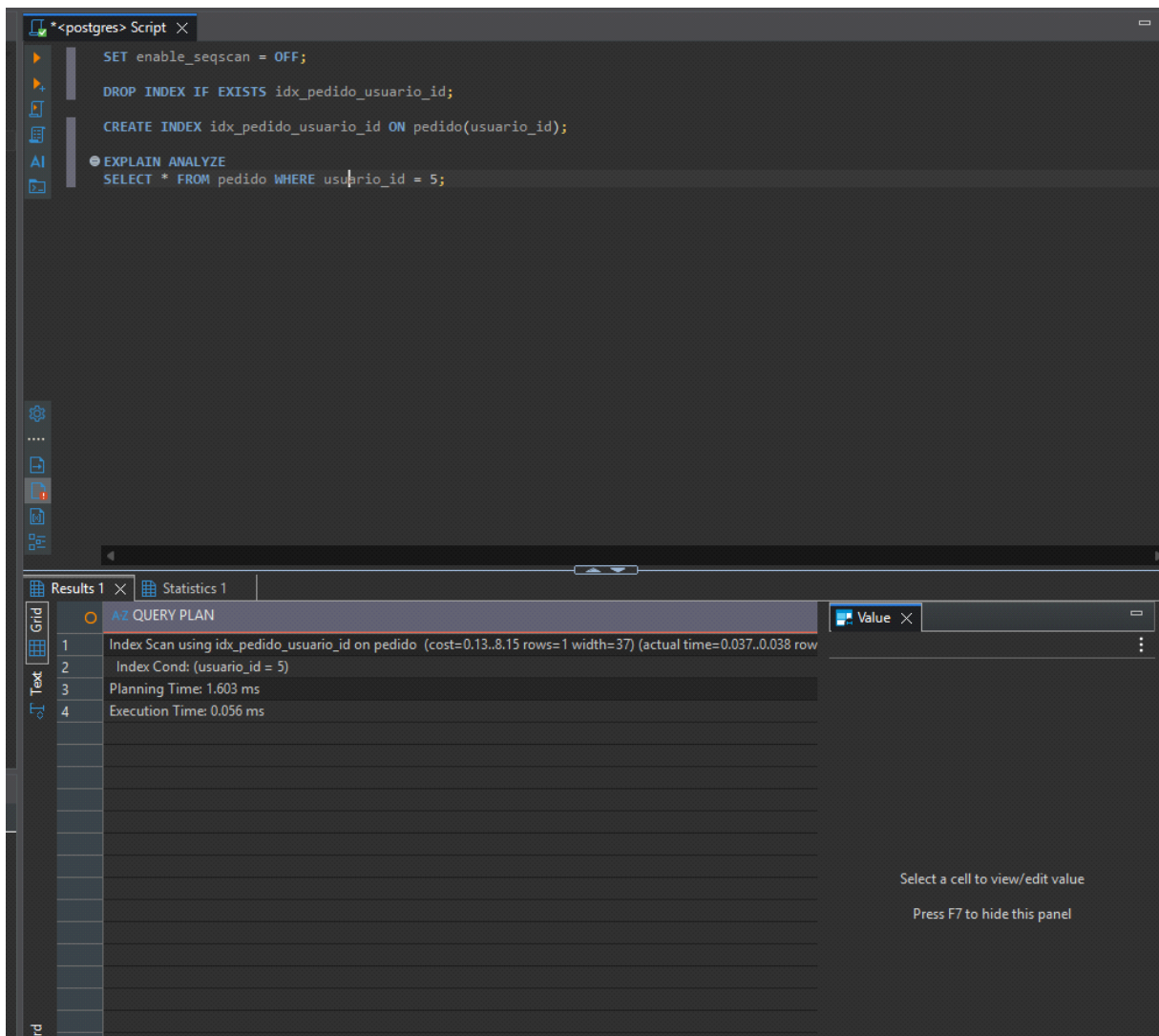
```
DROP INDEX IF EXISTS idx_pedido_usuario_id;  
  
EXPLAIN ANALYZE  
SELECT * FROM pedido WHERE usuario_id = 5;
```

The results panel displays the query plan for the second command. The plan is a Seq Scan on the 'pedido' table. The details are as follows:

| Step | Operation                 | Cost                              | Width | Actual Time | Actual Rows | Actual Loops |
|------|---------------------------|-----------------------------------|-------|-------------|-------------|--------------|
| 1    | Seq Scan on pedido        | (cost=0.00..1.09 rows=1 width=37) | 37    | 0.017       | 0.018       | 1            |
| 2    | Filter: (usuario_id = 5)  |                                   |       |             |             |              |
| 3    | Rows Removed by Filter: 6 |                                   |       |             |             |              |
| 4    | Planning Time: 0.745 ms   |                                   |       |             |             |              |
| 5    | Execution Time: 0.032 ms  |                                   |       |             |             |              |

The results panel also includes a 'Value' column and a 'Text' column. The 'Value' column contains the text 'Select a cell to view/edit value' and 'Press F7 to hide this panel'.

Después de crear el índice idx\_pedido\_usuario\_id (Index Scan):



Sin el índice, el plan usa Seq Scan on pedido con Execution Time = 0.032 ms, recorriendo las 7 filas de la tabla y descartando 6 por el filtro (Rows Removed by Filter: 6). Con el índice creado, el plan cambia a Index Scan using idx\_pedido\_usuario\_id, pero el Execution Time sube levemente a 0.056 ms (y el Planning Time también aumenta, de 0.745 ms a 1.603 ms).

Esto no es un error: para poder forzar el uso del índice en una tabla tan chica fue necesario ejecutar SET enable\_seqscan = OFF, porque el planner de PostgreSQL, por defecto, prefiere un Seq Scan cuando la tabla tiene pocas filas (en este caso, 7 registros de pedido). Con tan pocos datos, leer toda la tabla es más barato que consultar la estructura del índice y después ir a buscar la fila. El beneficio real de este índice se va a notar cuando la tabla pedido tenga muchos más registros (miles/millones), que es el escenario para el que están pensados los índices en un sistema en producción.

## 4.2 Consulta por nombre en producto (índice parcial)

Antes de crear el índice (Seq Scan):

The screenshot shows a PostgreSQL script editor with the following SQL commands:

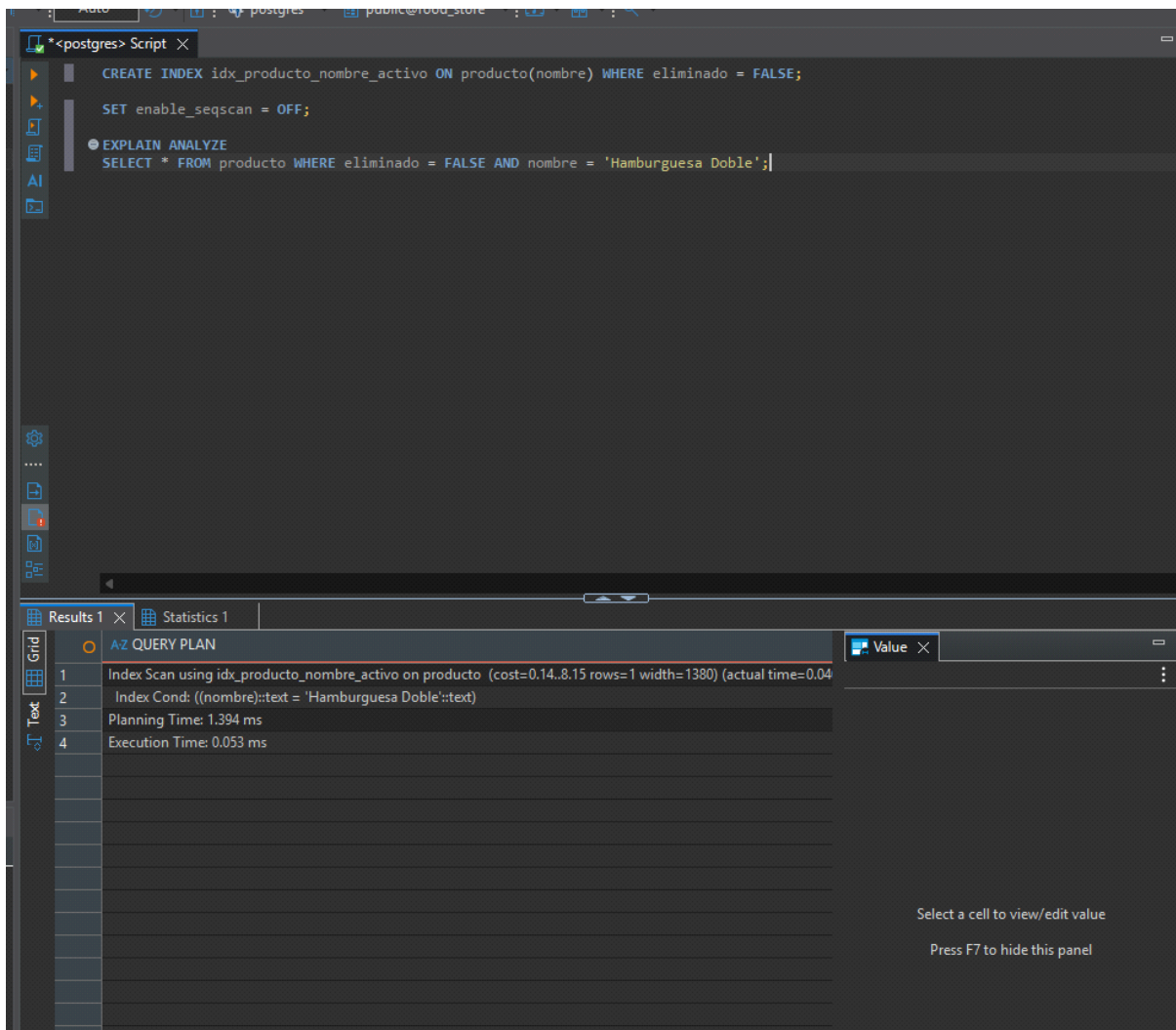
```
SET enable_seqscan = ON;  
DROP INDEX IF EXISTS idx_producto_nombre_activo;  
  
EXPLAIN ANALYZE  
SELECT * FROM producto WHERE eliminado = FALSE AND nombre = 'Hamburguesa Doble';
```

Below the script editor, the 'Results 1' tab is active, displaying the 'QUERY PLAN' for the executed query. The plan shows a sequential scan on the 'producto' table with a filter for 'Hamburguesa Doble'. The execution time is 0.029 ms.

| Step | Operation                                                                  | Cost                                | Rows | Width | Actual Time                               | Actual Rows | Actual Loops |
|------|----------------------------------------------------------------------------|-------------------------------------|------|-------|-------------------------------------------|-------------|--------------|
| 1    | Seq Scan on producto                                                       | (cost=0.00..1.21 rows=1 width=1380) | 1    | 1380  | (actual time=0.016..0.016 rows=0 loops=1) | 0           | 1            |
| 2    | Filter: ((NOT eliminado) AND ((nombre)::text = 'Hamburguesa Doble'::text)) |                                     |      |       |                                           |             |              |
| 3    | Rows Removed by Filter: 17                                                 |                                     |      |       |                                           |             |              |
| 4    | Planning Time: 0.932 ms                                                    |                                     |      |       |                                           |             |              |
| 5    | Execution Time: 0.029 ms                                                   |                                     |      |       |                                           |             |              |

On the right side of the results panel, there is a 'Value' tab and a message: 'Select a cell to view/edit value. Press F7 to hide this panel'.

Después de crear el índice parcial `idx_producto_nombre_activo` (Index Scan):



La consulta filtra por `eliminado = FALSE AND nombre = 'Hamburguesa Doble'` (un nombre que no existe literalmente en el catálogo, por lo que ambos planes devuelven 0 filas; lo relevante para el análisis es cómo se recorre la tabla para llegar a esa conclusión, no el resultado en sí). Sin índice, Seq Scan on producto recorre las 18 filas de la tabla y descarta 17 por el filtro (Execution Time = 0.029 ms). Con el índice parcial creado, el plan pasa a Index Scan using idx\_producto\_nombre\_activo, con Index Cond sobre nombre (Execution Time = 0.053 ms).

Este caso es un buen ejemplo de índice parcial (`WHERE eliminado = FALSE`): al indexar solo los productos vigentes, el índice queda más chico que uno sobre toda la tabla, lo que lo hace más rápido de mantener en escrituras y más liviano en memoria — un criterio de diseño explicado también en la sección 3.3 (Decisiones de diseño).

### 4.3 Conclusión general

En una base de datos con volumen de datos bajo (decenas de filas, como en este TPI de prueba), el optimizador de PostgreSQL suele preferir Seq Scan aun teniendo índices disponibles, porque el costo de acceso secuencial es mínimo. La utilidad real de los índices creados (idx\_pedido\_usuario\_id, idx\_producto\_nombre\_activo, idx\_producto\_categoria\_id, entre otros definidos en schema.sql) se evidencia con volúmenes de datos mayores, donde el plan Index Scan evita recorrer la tabla completa.

Por eso, en las pruebas se forzó SET enable\_seqscan = OFF: para poder comparar ambos planes de forma controlada aun con pocos datos.

## 5. Pruebas de concurrencia y bloqueos

### 5.1 Nivel READ COMMITTED (comportamiento por defecto)

| Paso | Sesión | Acción                                       | Resultado esperado                                  |
|------|--------|----------------------------------------------|-----------------------------------------------------|
| 1    | A      | BEGIN; primera lectura de stock              | stock = 12                                          |
| 2    | B      | UPDATE stock = stock - 5; COMMIT;            | stock confirmado = 7                                |
| 3    | A      | Segunda lectura de stock (misma transacción) | stock = 7 (ve el cambio de B → non-repeatable read) |

### 5.2 Nivel REPEATABLE READ

| Paso | Sesión | Acción                                                 | Resultado esperado                                                  |
|------|--------|--------------------------------------------------------|---------------------------------------------------------------------|
| 1    | A      | BEGIN ISOLATION LEVEL REPEATABLE READ; primera lectura | stock = 12                                                          |
| 2    | B      | UPDATE stock = stock - 5; COMMIT;                      | stock confirmado = 7                                                |
| 3    | A      | Segunda lectura (misma transacción)                    | stock = 12 (no ve el cambio de B → se evita el non-repeatable read) |

Lo que hay que capturar: las dos consultas de la Sesión A (Paso 1 y Paso 3) mostrando valores distintos en READ COMMITTED e iguales en REPEATABLE READ, más el UPDATE ya confirmado de la Sesión B en el medio. Idealmente, cuatro capturas (una por cada paso de lectura relevante) o una única captura con las dos terminales lado a lado.

## 6. Dificultades y soluciones

Lo que más nos costó de esta parte fue entender bien cómo funcionan las transition tables (referencing new table), ya que no es algo que se vea en los ejemplos básicos de triggers. También nos generó dudas la diferencia entre un trigger before y uno after, y por qué no se pueden juntar insert y update en el mismo trigger cuando se usa una transition table.

Para poder entenderlo recurrimos a varios videos de YouTube y a la documentación oficial, y fuimos probando con ejemplos chicos antes de aplicarlo en el proyecto. Por ejemplo, primero probamos armar un trigger simple que solo mostrara con RAISE NOTICE cuántas filas había en la tabla afectados, para entender que ahí quedan todas las filas insertadas de una sola sentencia (por ejemplo, si se cargan 3 detalles de pedido juntos, afectados tiene las 3 filas a la vez, no una por una).

Con eso más claro, entendimos que no hacía falta recalculer el total fila por fila: alcanzaba con sacar los pedido\_id distintos de afectados y actualizar el total de esos pedidos en un solo UPDATE. Y para lo de before/after, lo resolvimos probando manualmente: si el trigger de subtotal era after, el valor ya se

había guardado sin subtotal calculado, así que tuvo que quedar como before para poder modificar NEW antes del insert.

## 7. Bibliografía / Webgrafía

- W3Schools - SQL Tutorial: <https://www.w3schools.com/sql/>
- PostgreSQL Tutorial - Triggers: <https://www.postgresqltutorial.com/postgresql-triggers/>
- Documentación oficial de PostgreSQL - Triggers:  
<https://www.postgresql.org/docs/current/trigger-definition.html>
- Documentación oficial de PostgreSQL - CREATE VIEW:  
<https://www.postgresql.org/docs/current/sql-createview.html>
- Documentación oficial de PostgreSQL - CREATE FUNCTION:  
<https://www.postgresql.org/docs/current/sql-createfunction.html>

### Links

Repositorio: [nachoceba/TP\\_Concurrencia\\_IA\\_Grupo11: Grupo 11](#)